■ AI Project Mentor — Fix Summary Report

Date & Time: 2026-11-03 17:45:13
Session Goal / Focus: ■■■■■■■■■■■■ ■ ■■■■■■■ ■■■■ ■■■■ ■■■■■■■■■ ■■■■■■■■ ■ ■■■■
Total Fixes Applied: 6

---

1. Add default fallback for target path

Target File:	docker-compose.yml
Category:	improvement
Why / ROI:	Zero-configuration startup when launching the cloned repository, preventing docker compose mount errors.

Before (Original Code):

```
volumes:
- ${TARGET_PROJECT_PATH}:/workspace/project
# Mount local src directory for instant code updates
```

After (Applied Fix):

```
volumes:
- ${TARGET_PROJECT_PATH:-.}:/workspace/project
# Mount local src directory for instant code updates
```

2. Configure global Git safe directory

Target File:	Dockerfile
Category:	vulnerability
Why / ROI:	Enables reliable Git diffing, auto-commits, and pushes without permission crashes caused by host-container UID mismatches.

Before (Original Code):

```
# Install procps so pkill and kill commands are available for hot-reloading
RUN apt-get update && apt-get install -y --no-install-recommends procps openssh-client git && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
```

After (Applied Fix):

```
# Install procps so pkill and kill commands are available for hot-reloading
RUN apt-get update && apt-get install -y --no-install-recommends procps openssh-client git && rm -rf /var/lib/apt/lists/*

# Configure Git safe directory and system identity for containerized repository workflows
RUN git config --global --add safe.directory "*" && \
git config --global user.name "AI Project Mentor" && \
git config --global user.email "mentor@local"

COPY requirements.txt .
```

3. Scrub Git tokens and secret files

Target File:	src/scanner.py
Category:	vulnerability
Why / ROI:	Eliminates the danger of leaking GitHub personal access tokens, private keys, and runtime database files to external APIs or public Git remotes.

Before (Original Code):

```
IGNORE_DIRS = {
".git", ".venv", "venv", "env", "node_modules", "__pycache__",
".idea", ".vscode", "dist", "build", "migrations", "coverage"
}
```

```
IGNORE_FILES = {
".env", ".env.local", ".env.production", "package-lock.json",
"poetry.lock", "yarn.lock", "pnpm-lock.yaml"
}
ALLOWED_EXTENSIONS = {
".py", ".js", ".ts", ".tsx", ".jsx", ".html", ".css",
".md", ".json", ".yml", ".yaml", ".sql", ".sh", ".dockerfile"
}

SECRET_PATTERNS = [
re.compile(r'(?i)(api[_-]?key|secret|token|password|auth)\s*[:=]\s*["\']?[a-zA-Z0-9_\-\.:/+=]{12,}["\']?'),
re.compile(r'\b\d{8,10}:[a-zA-Z0-9_-]{35}\b'),
]
```

After (Applied Fix):

```
IGNORE_DIRS = {
".git", ".venv", "venv", "env", "node_modules", "__pycache__",
".idea", ".vscode", "dist", "build", "migrations", "coverage",
".pytest_cache", ".mypy_cache", ".ruff_cache"
}
IGNORE_FILES = {
".env", ".env.local", ".env.production", ".auth_token", "mentor.db",
"mentor.db-wal", "mentor.db-shm", "package-lock.json",
"poetry.lock", "yarn.lock", "pnpm-lock.yaml"
}
ALLOWED_EXTENSIONS = {
".py", ".js", ".ts", ".tsx", ".jsx", ".html", ".css",
".md", ".json", ".yml", ".yaml", ".sql", ".sh", ".dockerfile"
}

SECRET_PATTERNS = [
re.compile(r'(?i)(api[_-]?key|secret|token|password|auth)\s*[:=]\s*["\']?[a-zA-Z0-9_\-\.:/+=]{12,}["\']?'),
re.compile(r'\b\d{8,10}:[a-zA-Z0-9_-]{35}\b'),
re.compile(r'ghp_[a-zA-Z0-9]{36}'),
re.compile(r'github_pat_[a-zA-Z0-9_]{82}'),
re.compile(r'-----BEGIN [A-Z ]+ PRIVATE KEY-----[\s\S]*?-----END [A-Z ]+ PRIVATE KEY-----'),
]
```

4. Harden Git diff and status inspection

Target File:	src/main.py
Category:	improvement
Why / ROI:	Prevents bot crashes when pushing changes on new or uncommitted Git repositories.

Before (Original Code):

```
@dp.callback_query(F.data == "git:commit_push")
async def handle_git_commit_push(callback: CallbackQuery):
await callback.answer("■ Checking modified files in Git...")
proj_dir = config.CONTAINER_PROJECT_PATH

try:
# Allow git to operate inside the mounted directory
subprocess.run(["git", "config", "--global", "--add", "safe.directory", proj_dir], check=False)

# Get diff only for modified tracked files
diff_proc = subprocess.run(["git", "diff", "HEAD"], cwd=proj_dir, capture_output=True, text=True)
diff_text = diff_proc.stdout.strip()

if not diff_text:
await callback.message.answer("■■ No modified tracked files to commit (`git diff` is empty).")
return
```

After (Applied Fix):

```
@dp.callback_query(F.data == "git:commit_push")
async def handle_git_commit_push(callback: CallbackQuery):
await callback.answer("■ Checking modified files in Git...")
proj_dir = config.CONTAINER_PROJECT_PATH

try:
```

```
# Allow git to operate inside the mounted directory
subprocess.run(["git", "config", "--global", "--add", "safe.directory", proj_dir], check=False)

is_repo = subprocess.run(["git", "rev-parse", "--is-inside-work-tree"], cwd=proj_dir, capture_output=True, text=True)
if is_repo.returncode != 0:
await callback.message.answer("■■ Target directory is not a Git repository. Run <code>git init</code> first.", parse_mode="HTML")
return

diff_proc = subprocess.run(["git", "diff", "HEAD"], cwd=proj_dir, capture_output=True, text=True)
diff_text = diff_proc.stdout.strip() if diff_proc.returncode == 0 else ""
if not diff_text:
unstaged_diff = subprocess.run(["git", "diff"], cwd=proj_dir, capture_output=True, text=True)
diff_text = unstaged_diff.stdout.strip()

if not diff_text:
status_proc = subprocess.run(["git", "status", "--porcelain"], cwd=proj_dir, capture_output=True, text=True)
diff_text = status_proc.stdout.strip()

if not diff_text:
await callback.message.answer("■■ No modified files to commit (working tree is clean).")
return
```

5. Preserve author and safe Git config in patch apply

Target File:	src/main.py
Category:	improvement
Why / ROI:	Guaranteed automatic Git commits for applied micro-tasks without manual command-line intervention.

Before (Original Code):

```
if applied_files:
try:
for f in applied_files:
subprocess.run(["git", "add", str(f)], cwd=proj_dir, check=False)
commit_msg = f"refactor: {task['title'].lower()}"
subprocess.run(["git", "commit", "-m", commit_msg], cwd=proj_dir, check=False)
except Exception as e:
logging.debug(f"Git auto-commit failed: {e}")
```

After (Applied Fix):

```
if applied_files:
try:
subprocess.run(["git", "config", "--global", "--add", "safe.directory", str(proj_dir)], check=False)
has_user = subprocess.run(["git", "config", "user.name"], cwd=proj_dir, capture_output=True, text=True)
if not has_user.stdout.strip():
subprocess.run(["git", "config", "user.name", "AI Project Mentor"], cwd=proj_dir, check=False)
subprocess.run(["git", "config", "user.email", "mentor@local"], cwd=proj_dir, check=False)
for f in applied_files:
subprocess.run(["git", "add", str(f)], cwd=proj_dir, check=False)
commit_msg = f"refactor: {task['title'].lower()}"
subprocess.run(["git", "commit", "-m", commit_msg], cwd=proj_dir, check=False)
except Exception as e:
logging.debug(f"Git auto-commit failed: {e}")
```

6. Document Git preparation and environment variables

Target File:	README.md
Category:	improvement
Why / ROI:	Clear repository setup steps for collaborators, preventing accidental commits of secret tokens and missing dependencies.

Before (Original Code):

```
## Setup
1. Clone the repository.
2. Create a `.env` file based on `.env.example`.
```

```
3. Run `docker-compose up --build`.

## Usage
- Send `/start` to the Telegram bot to register.
- Use `/scan` to analyze your project.
- Use `/next` to get your next task.
```

After (Applied Fix):

```
## Setup
1. Clone the repository: `git clone <repo-url> && cd ai-project-mentor`
2. Create a `.env` file with `GEMINI_API_KEY`, `TELEGRAM_BOT_TOKEN`, and optional `TARGET_PROJECT_PATH`.
3. Verify `.env` is listed in your `.gitignore` before committing.
4. Run `docker-compose up --build`.

## Usage
- Send `/start` to the Telegram bot to register.
- Use `/scan` or `/focus` to analyze your project and find improvements.
- Use `/next` to receive and apply micro-tasks.
- Tap **Commit & Push Changes** to record fixes to your Git history.
```